- Big-O, Big-Omega, Big-Theta:
  - $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) \ldots < O(1.1^n) < O(2^n) \ldots$
  - Big-O
    - upper bound
    - $f(n) = O(g(n))$

      $\Rightarrow g(n)$ is the upper bound of $f(n)$
    - $n^3 = O(n^2)$ ✗

      $4n^3 = O(n^3)$ ✓

      $3n^2 + 7n = O(1.1^n)$ ✓

$\Omega(n \log n)$ — $n \log n$ — $O(n \log n)$

$\Omega(n)$ — $O(n^2)$

$\Omega(\log n) \ldots$ — $O(n^3) \ldots$

  - Big-Omega
    - Ω
    - Lower bound
    - Exact opposite of Big O
    - $f(n) = \Omega(g(n))$

      $\Rightarrow g(n)$ is the lower bound of $f(n)$
    - $n^3 = \Omega(n^2)$ ✓

      $n^3 = \Omega(7n^3)$ ✓

      $4n^3 = \Omega\left(\frac{n^4}{100}\right)$ ✗
  - Big-Theta
    - Exact Bound
    - $f(n) = \Theta(g(n))$

      $\Rightarrow g(n)$ is both the upper bound & also the lower bound of $f(n)$

      $\Rightarrow \boxed{f(n) = O(g(n)) \text{ AND } f(n) = \Omega(g(n))}$
    - $7n^2 = \Theta(n^2)$ ✓

      $7n^2 = O(n^2)$ ✓

      $7n^2 = \Omega(n^2)$ ✓
    - $7n^3 = \Theta(n^4)$ ✗

      $7n^2 = O(n^4)$ ✓

      $7n^2 = \Omega(n^4)$ ✗

Algo1: $7n^2 + 4n + 3 =$ $O(n^3)$ ✓, $O(n^2)$ ✓, $\Theta(n^2)$ ✓, $\Theta(n^3)$ ✗

- Worst case analysis
  - given an array, look for a particular query element.

```
boolean isExists( int[]arr, int key) {
    for( int x : arr) {
        if( x == key)
            return true;
    }
    return false;
}
```

- If the first element is itself the key ⇒ 'k' steps = $\Theta(1)$ constant time
- If the key doesn't exist in the array ⇒ 'kn+p' steps = $\Theta(n)$ linear time

Worst case

Best case

$O(n)$ $\Omega(n)$

```
1 — int result = 0;
n^2 { for (int i=0; i<n; i++) {
    n { for (int j=0; j<n; j++) {
          result += i*j;
      }
    }
1 — return result;
```

$\Theta(n^2)$

```
// 1 step
// for each i iteration
    // i<n          1 step
    // j loop
        // j<n      1 step
        // i*j      1 step
        // result+= 2 steps
        // j++      1 step
    // i++          1 step
// return result    // 1 step
```

5n steps

Total = $(1 + 5n + 1)n + 1 + 1$

$= (5n+2)n + 2$

$= 5n^2 + 2n + 2$ steps

$O(n^2)$ $\Omega(n^2)$ $\Theta(n^2)$

$O(n^3)$ $\Omega(n \log n)$

$O(n^4) \ldots$ $\Omega(n)$

```
for (int i=0; i<n; i++) {
    for (j=0; j<i; j++) {
        print("Hi");
    }
}
```

	Inner loop runs
$i=0$	0
$i=1$	1
$i=2$	2
$\vdots$	$\vdots$
$i=n-1$	$n-1$

$$0+1+\cdots+n-1 = \frac{n(n-1)}{2} = \Theta(n^2)$$

```
for (i=0; i<n; i++) {
    for (j=0; j<10000; j++) {
        print(..)
    }
}
```

Total steps $= 10000n = \Theta(n)$

```
for (i=0; i<n; i++) {
    for (j=0; j<n; j++) {
        for (k=0; k<j; k++) {
            print(..)
        }
    }
}
```

n

$$\frac{n(n-1)}{2} = \Theta(n^2)$$

Total $: n \times \frac{n(n-1)}{2} = \Theta(n^3)$

$= n \times \Theta(n^2) = \Theta(n^3)$

```
for (int i=n; i>0; i=i/2) {
    print("Hi");
}
```

$\log_2 n$

$i=n$	- 1 step
$i=n/2$	- 1
$\vdots$	
$i=1$	- 1 step
$i=0$	- X

times this loop runs $= \log_2 n + 1$ times

$= \Theta(\log n)$

$\boxed{n=16}$

$i=16$	Hi
$i=8$	Hi
$i=4$	Hi
$i=2$	Hi
$i=1$	Hi
$i=0$	X

$5 = \log_2 16 + 1$

$n \downarrow 2$, $n/2 \downarrow 2$, $n/4$, $\vdots$, $\downarrow 1$ — $\log_2 n$

```
for (i=0; i<n; i++) {
    for (j=n; j>1; j=j/2) {
        print("Hi")
    }
}
```

log n times

$\Theta(n \log n)$

1, 3, 7, 2

$1 < 3$

$3 < 7$

$7 < 2$ ✗

Sorting (Selection sort)

- Best case scenario is if array is already sorted $= \boxed{\Theta(n) \text{ time}}$
- Worst case scenario array is not sorted $= \Theta(n^2)$ time

$S = 1 + 2 + 3 + \cdots + n$

$S = n + (n-1) + (n-2) + \cdots + 1$

$2S = (n+1) + (n+1) \cdots + (n+1)$

$2S = n(n+1)$

$\boxed{S = \frac{n(n+1)}{2}}$ = sum of first 'n' natural numbers

$0 + 1 + 2 + \cdots + n-1$

= sum of first 'n-1' natural numbers

$= \frac{(n-1)(n-1+1)}{2}$

$= \frac{n(n-1)}{2}$

Linear search

- Best case scenario: when query element is the first element itself

  RT: $\Theta(1)$

- Worst case scenario: when query element is not present in the array

  RT: $\Theta(n)$

Selection sort

- Best: array is sorted

  RT: $\Theta(n)$

- Worst: not sorted

  RT: $\Theta(n^2$

Binary Search

- Array has to be sorted for it to work
-

0	1	2	3	4	5	6	7	8	9	10
0	7	11	12	14	16	18	20	25	91	107

key = 13

$m = (l+h)/2 = 5$

$m = (0+4)/2 = 2$

$m = (3+4)/2 = 3$

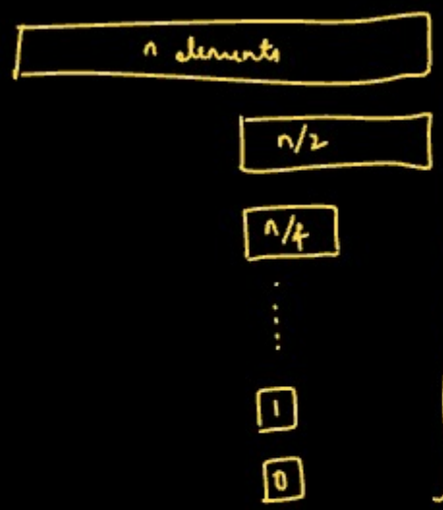
$m = (4+4)/2 = 4$

	Best case RT	Worst case RT
Linear Search	$\Theta(1)$	$\Theta(n)$
Binary Search	$\Theta(1)$	$\Theta(\log n)$

Binary Search → array has to be in sorted order

$\Theta(1)$ → found in first level itself

$\Theta(n)$, $\Theta(\log n)$ → when element not present



'k' steps { find m
compare query elem with middle
$l = m+1$ or $h = m-1$

#levels = $\log_2 n + 2$ in the worst case

Total RT = $(\log_2 n + 2)k$
$= \Theta(\log n)$

16 ↓ 8 ↓ 4 ↓ 2 ↓ 1 ↓ 0

Sorting

- Insertion sort
- Selection sort
- Bubble sort

} $\Theta(n^2)$

- Merge sort
- Quick sort
- Heap sort

} $\Theta(n \log n)$

- Radix sort
- Counting sort..

} We need info about the array (faster than $n \log n$)

- Stability

$A_1\ B_1\ A_2\ C_1\ B_2$

$A_1 A_2\ B_1 B_2\ C_1$ (Sorted) ✓ → Stable

$A_2 A_1\ B_1 B_2\ C$ (Sorted) → Not stable (as A_1 came before A_2 in original but A_1 came after A_2 in sorted)

- Inplace algo : that doesn't use extra space

Sorting

Bubble sort – Inplace? Yes

11, -1, 5, 6, 3, -2, 4
-1, 11, 5, 6, 3, -2, 4
-1, 5, 11, 6, 3, -2, 4
-1, 5, 6, 11, 3, -2, 4
-1, 5, 6, 3, 11, -2, 4
-1, 5, 6, 3, -2, 11, 4
-1, 5, 6, 3, -2, 4, 11 } 1st pass

-1, 5, 3, 6, -2, 4, 11
-1, 5, 3, -2, 6, 4, 11
-1, 5, 3, -2, 4, 6, 11 } 2nd pass

-1, 3, 5, -2, 4, 6, 11
-1, 3, -2, 5, 4, 6, 11
1, 3, -2, 4, 5, 6, 11 } 3rd pass

-1, -2, 3, 4, 5, 6, 11 } 4th pass

-2, -1, 3, 4, 5, 6, 11 } 5th pass

Bubble sort

· Selection sort – Inplace? Yes!

11, -1, 5, 6, 3, -2, 4
i min

-2 -1, 5, 6, 3, 11, 4
i min

-2 -1 5, 6, 3, 11, 4
i min

-2 -1 3 6 5 11 4
i min

-2 -1 3 4 5 11 6
i min

-2 -1 3 4 5 11 6
i min

-2 -1 3 4 5 6 11
i

Bubble sort

1 4 8 0 5
1 4 0 8 5
1 4 0 5 8 } 1st pass

1 0 4 5 8 } 2nd pass

0 1 4 5 8 } 3rd pass

5 4 3 2
4 5 3 2
4 3 5 2
4 3 2 5 } 1st pass

3 4 2 5
3 2 4 5 } 2nd pass

2 3 4 5 } 3rd pass

- Insertion sort — Inplace? Yes!

[ ] 11, -1, 5, 6, 3, -2, 4

↑ i

[11] -1 5 6 3 -2 4

↑ i

[-1 11] 5 6 3 -2 4

↑ i

[-1 5 11] 6 3 -2 4

↑ i

[-1 5 6 11] 3 -2 4

↑ i

[-1 3 5 6 11] -2 4

↑ i

[-2 -1 3 5 6 11] 4

↑ i

[-2 -1 3 4 5 6 11]

1 7 4 11 0

Bubble sort:

1 7 4 11 0
1 4 7 11 0
1 4 7 0 $\boxed{11}$ } 1st pass

1 4 0 $\boxed{7 \quad 11}$ } 2nd pass

1 0 $\boxed{4 \quad 7 \quad 11}$ } 3rd pass

0 $\boxed{1 \quad 4 \quad 7 \quad 11}$ } 4th pass

5 1 2 3 4
1 5 2 3 4
1 2 5 3 4
1 2 3 5 4
1 2 3 4 5 } 1st pass

1 2 3 4 5 } 2nd pass

If in a particular pass if no swaps occur ⇒ array is sorted

Selection sort

1 7 4 11 0
↑ i ↑ min

0 7 4 11 1
↑ i ↑ min

0 1 4 11 7
↑ i ↑ min

0 1 4 11 7
↑ i ↑ min

0 1 4 7 11
↑ i

Insertion sort

[] 1 7 0 11 4
↑ i

$\boxed{1}$ 7 0 11 4
↑ i

$\boxed{1 \quad 7}$ 0 11 4
↑ i

$\boxed{0 \quad 1 \quad 7}$ 11 4
↑ i

$\boxed{0 \quad 1 \quad 7 \quad 11}$ 4
↑ i

$\boxed{0 \quad 1 \quad 4 \quad 7 \quad 11}$

Running time

· Selection Sort (n elements)

Best case

$i=0$ #comparisons $= n-1$ #swaps $= 0$
$i=1$ #comparisons $= n-2$ #swaps $= 0$
⋮
$i=n-2$ #comparisons $= 1$ #swaps $= 0$
$i=n-1$ #comparisons $= 0$ #swaps $= 0$

#Comparisons = $(n-1)+(n-2)+\cdots+1+0$ #swaps = 0

$= \frac{n(n-1)}{2} = \frac{n^2}{2} - \frac{n}{2}$

Total work = #comparisons + #swaps

$= \frac{n^2}{2} - \frac{n}{2} = \Theta(n^2)$

worst case

Total #comparisons: $\frac{n(n-1)}{2}$

Total #swaps $= n-1$ (one for every i from 0 to $n-2$)

$(n-1)\left(\frac{n}{2}+1\right) = \frac{(n-1)(n+2)}{2}$

$= \Theta(n^2)$

Insertion sort

$\boxed{1 \quad 3 \quad 6}$ 7

- Best case

$i=0$	# comparisons: 0	# swaps = 0
$i=1$	# comparisons: 1	# swaps = 0
$\vdots$	$\vdots$	$\vdots$
$i=n-2$	# comparisons: 1	# swaps = 0
$i=n-1$	# comparisons: 1	# swaps = 0

Total # comp : $n-1$ Total # swaps = 0

Total RT = $\Theta(n)$

- Worst case:

$\boxed{2 \quad 3 \quad 4}$ 1

$i=0$	# comp = 0	# swaps = 0
$i=1$	# comp : 1	# swaps = 1
$i=2$	# comp : 2	# swaps : 2
$\vdots$		
$i=n-1$	# comp : $n-1$	# swaps = $n-1$

Total # comp : $\frac{n(n-1)}{2}$ # swaps : $\frac{n(n-1)}{2}$

Total RT : $n(n-1) = \Theta(n^2)$

Stability:

- A stable sorting algo preserves the relative ordering of duplicate elements.

$B_1, A_1, C, B_2, B_3, A_2$

$\downarrow$ stable sort

$A_1 \; A_2 \; B_1 \; B_2 B_3 \; C$

- Long distance exchanges
  $\Rightarrow$ NOT STABLE

$B_1 \; B_2 \; C \; A$

$\uparrow_i \qquad \uparrow_{min}$

$A \; B_2 \; C \; B_1$

$\uparrow_i$

Selection sort not stable (counter example)

$\boxed{A_1 \; B_1 \; C} \; A_2$

$A_1 \; B_1 \; A_2 \; C$

$A_1 \; A_2 \; B_1 \; C$

Insertion (stable)

	Best Case RT	Worst Case RT	Stable?	In place?
Insertion sort	$\Theta(n)$	$\Theta(n^2)$	✓	✓
Selection sort	$\Theta(n^2)$	$\Theta(n^2)$	✗	✓
Bubble sort	$\Theta(n)$	$\Theta(n^2)$	✓	✓

Bubble sort

- Best case scenario: Already sorted

  # comparisons $= n-1$ (for 1 pass)

  # swaps $= 0$

- RT: $\Theta(n)$

- Worst case:
  - Reverse order
  -

	# comparisons	# swaps
1st pass	$n-1$	$n-1$
2nd pass	$n-2$	$n-2$
⋮	⋮	⋮
$(n-1)$th pass	1	1
	Total: $\frac{n(n-1)}{2}$	$+ \frac{n(n-1)}{2}$

RT: $n(n-1) = \Theta(n^2)$

Inversions

- A pair of elements that is out of order is called an <u>inversion</u>

4, 3, 1, 7, 9, -5

(4, 3)
(4, 1)
(4, -5)
(3, 1)
(3, -5)
(1, -5)
(7, -5)
(9, -5)

$\Rightarrow$ # inversions = 8

- # swaps in insertion sort = # inversions

$\boxed{4}$ 3, 1, 7, 9, -5 — (3, 4)

$\boxed{3, 4}$ 1, 7, 9, -5 — (4, 1) (3, 1)

$\boxed{1, 3, 4}$ 7 9 -5

$\boxed{1\ 3\ 4\ 7}$ 9 -5 — (9, -5) (7, -5) (4, -5) (3, -5) (1, -5)

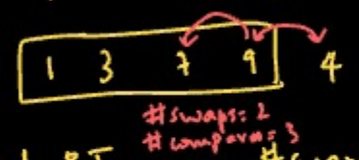
$\boxed{1\ 3\ 4\ 7\ 9}$ -5

$\boxed{-5\ 1\ 3\ 4\ 7\ 9}$

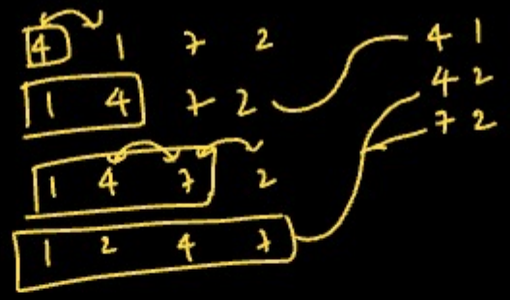
For insertion sort

- #swaps = #inversions

  #comparisons = #swaps + (n-1)



	#compares
i=1	#swaps + 1
i=2	#swaps + 1
⋮	
i=n-1	#swaps + 1
	#swaps + (n-1)



- Total RT of insertion sort = #swaps + #compares

  = #inversions + (#inversions + (n-1))

  = 2(#inversions) + (n-1)

Partially sorted arrays

- Are arrays where #inversions $\leq cn$

- RT for insertion sort for partially sorted arrays $= 2(cn) + (n-1)$

  $= (2c+1)n - 1$

  $= \Theta(n)$

$1 + 2 + \cdots + n = \Theta(n^2)$ $\qquad \frac{n(n+1)}{2}$

$1 + 2 + \cdots + n-1 = \Theta(n^2)$ $\qquad \frac{n(n-1)}{2}$

$(n-1) + (n-2) + \cdots + 2 + 1 = \Theta(n^2)$

$n + (n-1) + \cdots + 1 = \Theta(n^2)$

$n + n + \cdots + n = \Theta(n^2)$

$(n+1) + (n+1) + \cdots + (n+1) = \Theta(n^2)$

$$n^2 = (n) + \underbrace{(1+2+\cdots+(n-1))}_{S} + \underbrace{(1+2+\cdots+(n-1))}_{S}$$

$$= n + S + S$$

$$n^2 = n + 2S \Rightarrow 2S = n^2 - n$$

$$\Rightarrow S = \frac{n^2-n}{2}$$

$$\boxed{S = \frac{n(n-1)}{2}}$$